

AUTOSAR Classic Platform Release R25-11 | Document ID 53

Document Title: Layered Software Architecture

Section: 1.0 Specification Header

Document Title: Layered Software Architecture

Document Owner: AUTOSAR

Document Responsibility: AUTOSAR

Document Identification No: 53

Document Status: published

Part of AUTOSAR Standard: Classic Platform

Part of Standard Release: R25-11

Document ID: AUTOSAR_CP_EXP_LayeredSoftwareArchitecture

AUTOSAR Classic Platform Release R25-11 | Document ID 53

Document Change History: Releases R25-11 & R24-11

Section: Document Change History

Document Change History

Release R25-11 (2025-11-27): Added information about VDP, Mirror. Updated slides about libraries, DDS. Removed TTCan, Fls and Eep, LdCom.

Release R24-11 (2024-11-27): Added L-SDU Router. Incorporated J1939Fscp transformer into comm stack extensions. Incorporated Partitioning examples. Incorporated migration of BSWModuleList to BSWGeneral. Added I2C in Comm Drivers. Added changes in memory manipulation library: Copy, Set, Move, Compare. Removed E2EPW support.

Document Change History (Continued)

Release R23-11 (2023-11-23): Added information about charging management (ChrgM) and firewall.

Release R22-11 (2022-11-24): Incorporated new concepts for Vehicle-2-X Data Manager, MACsec, CAN XL, DDS, Secured Time Synchronization, Vehicle-2-X Support for China.

Release R21-11 (2021-11-25): Incorporated draft concept for new Memory Driver and Memory Access.

Release R20-11 (2020-11-30): Removed Pretended Networking. Added caveats for E2E Protection Wrapper. Layer Interaction Matrix: Allow Crypto Driver to access Memory Services. Incorporated new concepts for Intrusion Detection System Manager, CP Software Clusters.

Release R19-11 (2019-11-28): Incorporated new concepts for Atomic multicore safe operations, Signal-service-translation, NV data handling enhancement.

Table of contents

1. Architecture

1.1 Overview of Software Layers

1.2 Content of Software Layers

1.3 Content of Software Layers in Multi-Core Systems

1.4 Content of Software Layers in Mixed-Critical Systems

1.5 Overview of Modules

1.6 Interfaces: General Rules

1.7 Interfaces: Interaction of Layers

1.8 Overview of CP Software Clusters

2. Configuration

3. Integration and Runtime Aspects

Introduction: Purpose and Inputs

Section: 1.0 Purpose and Inputs

Introduction: Purpose of this document

The Layered Software Architecture describes the software architecture of AUTOSAR:

- It describes in a top-down approach the hierarchical structure of AUTOSAR software.
- Maps the Basic Software Modules (BSW) to software layers and shows their relationship.
- This document focuses on static views of a conceptual layered software architecture.
- Maps BSW modules: Services, ECU Abstraction, Microcontroller Abstraction, and Complex Drivers.

Introduction: Scope and Extensibility

Section: 1.0 Scope and Extensibility

Introduction: Application scope of AUTOSAR

AUTOSAR is dedicated for Automotive ECUs with:

- Strong interaction with hardware (sensors and actuators)
- Connection to vehicle networks like CAN, LIN, FlexRay, or Ethernet
- Microcontrollers (typically 16 or 32 bit) with limited computing power and memory
- Real Time System and program execution from internal or external flash memory.

NOTE: In the AUTOSAR sense, an ECU means one microcontroller plus peripherals and the according software/configuration.

AUTOSAR Extensibility: Standard modules can be extended in functionality. Non-standard modules can be integrated into AUTOSAR-based systems as Complex Drivers. Further layers cannot be added.

Architecture: Overview of Software Layers - Top view

The AUTOSAR Architecture distinguishes on the highest abstraction level between three software layers:

1. Application Layer (Application Software Components)
2. Runtime Environment (RTE)
3. Basic Software (BSW)

These layers execute on top of the underlying Microcontroller hardware.

Architecture: Overview of Software Layers - Coarse view

The AUTOSAR Basic Software (BSW) is further divided into:

1. Services Layer (Highest layer of BSW)
2. ECU Abstraction Layer
3. Microcontroller Abstraction Layer (MCAL, lowest layer)
4. Complex Drivers (Spanning from hardware to the RTE)

Architecture: Detailed view of Basic Software Layers

The Basic Software Layers are divided into functional groups:

- System Services: Operating System, Error Management, State Management
- Memory Services: NVRAM Management (NvM)
- Crypto Services: Crypto Service Manager (CSM), Key Manager (KeyM), Intrusion Detection System Manager (IdsM)
- Off-board Communication Services: V2X Data Manager
- Communication Services: AUTOSAR COM, PDU Router (PduR), IPDU Multiplexer, Diagnostic Communication Manager (Dcm)
- I/O Hardware Abstraction: I/O Signal Interface, Drivers for external ADC/IO ASICs
- Memory Hardware Abstraction: Memory Abstraction Interface (MemIf), Fee, Ea
- Crypto Hardware Abstraction: Crypto Interface, Crypto Drivers
- Communication Hardware Abstraction: CAN Interface (CanIf), LIN Interface (LinIf), FlexRay Interface (FrIf), Ethernet Interface (EthIf), L-SDU Router
- Microcontroller Drivers (MCAL): GPT Driver, Watchdog Driver, MCU Driver, Core Test, Internal Flash Driver, RAM Test, SPI Handler Driver, LIN Driver, CAN Driver, FlexRay Driver, Ethernet Driver, I2C Driver, OCU Driver, ICU Driver, PWM Driver, ADC Driver, DIO Driver, PORT Driver.

Architecture: Microcontroller Abstraction Layer (MCAL)

The Microcontroller Abstraction Layer is the lowest software layer of the Basic Software.

It contains internal drivers, which are software modules with direct access to the microcontroller and internal peripherals.

Task: Make higher software layers independent of the microcontroller (uC).

Properties:

- Implementation: uC dependent.
- Upper Interface: Standardized and uC independent.

Architecture: ECU Abstraction Layer

The ECU Abstraction Layer interfaces the drivers of the Microcontroller Abstraction Layer. It also contains drivers for external devices.

It offers an API for access to peripherals and devices regardless of their location (uC internal/external) and their connection to the uC (port pins, type of interface).

Task: Make higher software layers independent of ECU hardware layout.

Properties:

- Implementation: uC independent, ECU hardware dependent.
- Upper Interface: uC and ECU hardware independent.

Architecture: Complex Drivers

The Complex Drivers Layer spans from the hardware to the RTE.

Task: Provide the possibility to integrate special purpose functionality, e.g. drivers for devices:

- Which are not specified within AUTOSAR
- With very high timing constraints (e.g. injection control, electric valve control, incremental position detection)
- For migration purposes.

Properties:

- Implementation: Highly uC, ECU, and application dependent.
- Upper Interface to SW-Cs: Specified and implemented according to AUTOSAR (AUTOSAR interface).
- Lower Interface: Restricted access to Standardized Interfaces.

Architecture: Services Layer

The Services Layer is the highest layer of the Basic Software. While access to I/O signals is covered by the ECU Abstraction Layer, the Services Layer offers:

- Operating system functionality (AUTOSAR OS)
- Vehicle network communication and management services (ComM, CanSM, LinSM, FrSM, EthSM, Nm)
- Memory services (NVRAM management via NvM)
- Diagnostic Services (UDS communication via Dcm, error memory and fault treatment via Dem, FiM, Det, Dlt)
- ECU state management, mode management (EcuM, BswM)
- Logical and temporal program flow monitoring (Watchdog Manager - WdgM)

Task: Provide basic services for applications, RTE, and basic software modules.

Architecture: AUTOSAR Runtime Environment (RTE)

The RTE is a layer providing communication services to the application software (AUTOSAR Software Components and/or AUTOSAR Sensor/Actuator components).

Above the RTE the software architecture style changes from 'layered' to 'component style'.

The AUTOSAR Software Components communicate with other components (inter- and/or intra-ECU) and/or services via the RTE.

Task: Make AUTOSAR Software Components independent from the mapping to a specific ECU.

Properties:

- Implementation: ECU and application specific (generated individually for each ECU).
- Upper Interface: Completely ECU independent.

Architecture: Basic Software Module Types - Manager

A manager offers specific services for multiple clients. It is needed in all cases where pure handler functionality is not enough to abstract from multiple clients.

Besides handler functionality, a manager can evaluate and change or adapt the content of the data.

In general, managers are located in the Services Layer.

Example: The NVRAM Manager (NvM) manages concurrent access to internal and/or external memory devices like flash and EEPROM. It also performs distributed reliable data storage, data checking, and provision of default values.

Architecture: Communication Services - General

The Communication Services are a group of modules for vehicle network communication (CAN, LIN, FlexRay, and Ethernet). They interface with the communication drivers via communication hardware abstraction.

Modules:

- AUTOSAR COM: Signal gateway, routes individual signals between I-PDUs.
- PDU Router (PduR): Provides routing of PDUs between different communication controllers and upper layers.
- IPDU Multiplexer (IpduM): Multiplexes I-PDUs on bus channels.
- Diagnostic Communication Manager (Dcm): Handles UDS and OBD diagnostics.
- Diagnostic Log and Trace (Dlt): Standardized logging and tracing.
- Generic NM Interface & Network Management modules: CanNm, LinNm, FrNm, UdpNm.

Architecture: Communication Stack - Firewall

The Firewall module protects the AUTOSAR stack from malicious messages by inspecting network packets and filtering them based on a pre-defined ruleset.

The firewall supports network packet inspection on 3 different levels:

1. Stateless packet inspection
2. Stateful packet inspection
3. Deep packet inspection

The firewall is connected to the IdsM (Intrusion Detection System Manager) module to raise security events in the case of unexpected network packets.

Interactions:

Firewall -> IdsM (Security events)

BswM -> Firewall (Firewall state management)

Architecture: Communication Stack - Charging Manager (ChrgM)

The Charging Manager (ChrgM) belongs to Communication Services of the AUTOSAR Layered Architecture.

Tasks:

- ChrgM controls the charging process between the EV (Electric Vehicle) and the EVSE (Electric Vehicle Supply Equipment) as per ISO 15118-2.
- ChrgM communicates with different BSW modules: PduR, SoAd, Csm, KeyM, BswM to enable the charging process.
- Provides ports used by application SWCs implementing EV charging.
- Submodules: V2GTP (Vehicle to Grid standard protocol) and EXI (Efficient XML Interchange).

Architecture: Overview of Modules - Implementation Conformance Class 3 (ICC3)

Complete mapping of standard AUTOSAR Basic Software modules to layers:

- System Services: Dem (Diagnostic Event Manager), FiM (Function Inhibition Manager), Det (Default Error Tracer), StbM (Synchronized Time-base Manager), Tm (Time Service), WdgM (Watchdog Manager), ComM (Communication Manager), BswM (Basic Software Mode Manager), EcuM (ECU State Manager), AUTOSAR OS.
- Memory Services: NvM (NVRAM Manager).
- Communication Services: Com, Dcm, Dlt, SecOC, IpduM, PduR, Xf (Transformers), Tp (Transport Protocols).
- I/O Hardware Abstraction: I/O Signal Interface, External ASIC drivers.
- Memory Hardware Abstraction: MemIf, Fee (Flash EEPROM Emulation), Ea (EEPROM Abstraction), MemAcc.
- Onboard Device Abstraction: WdgIf.
- Communication Hardware Abstraction: CanIf, LinIf, FrIf, EthIf, L-SDU Router.
- MCAL Drivers: Gpt, Wdg, Mcu, CorTst, FlsTst, RamTst, Mem, Spi, Lin, Can, Fr, Eth, I2C, Ocu, Icu, Pwm, Adc, Dio, Port.

Interfaces: Type of Interfaces in AUTOSAR

1. AUTOSAR Interface: Defines information exchanged between software components and/or BSW modules. Independent of programming language, ECU or network technology. Used in defining ports (P-Ports, R-Ports).
2. Standardized AUTOSAR Interface: An AUTOSAR Interface whose syntax and semantics are standardized within AUTOSAR. Typically used to define AUTOSAR Services provided by BSW to application SWCs.
3. Standardized Interface: An API standardized within AUTOSAR without using the AUTOSAR Interface port technique (C-APIs, e.g., MemIf_Write, Dem_SetEventStatus). Restricted to intra-ECU module communication.

Normative rules governing vertical layer calls:

- SW Components / RTE are allowed to use: System Services, Memory Services, Crypto Services, Communication Services, Off-board Communication Services, Complex Drivers, I/O Hardware Abstraction.
- SW Components are NOT allowed to bypass RTE to call MCAL drivers directly.
- Bypassing the Microcontroller Abstraction Layer is strictly forbidden.
- I/O Drivers are allowed to use System Services and Hardware, but no other layers.

Background: Comparison between memory service modules and memory types

1. NvM (NVRAM Manager): Storage of module data (error info, configuration, status). Supports high-frequency parallel read/write across BSW and SW-C. Buffers data in RAM.
2. BndM (Bulk NV Data Manager): Storage of large vehicle-specific data written very infrequently (repair shop diagnostics). Users have direct access via pointer (BndM_GetBlockPtr).
3. FOTA Manager: Storage of model-specific vehicle firmware/code update packages. Typical size in Megabytes. Supports background read-while-write over driving cycles.

Interfaces: Interaction of Layers - Example Memory

Interaction between NVRAM Manager and hardware:

- NvM interacts with Memory Abstraction Interface (MemIf).
- MemIf routes requests via Fee (Flash EEPROM Emulation) or Ea (EEPROM Abstraction).
- Fee accesses internal Flash driver. Ea accesses external EEPROM driver via SPIHandlerDriver.
- Watchdog Manager (WdgM) accesses external Watchdog Driver via SPIHandlerDriver with higher priority than EEPROM.

Interfaces: Interaction of Layers - Example Communication Stack

Comprehensive communication interaction architecture:

- Application SWCs send signals through RTE to AUTOSAR COM.
- COM serializes signals into I-PDUs and transfers them to PDU Router (PduR).
- PduR routes I-PDUs to bus transport protocols (CanTp, J1939Tp, FlexRay Tp) or directly to bus interfaces (CanIf, LinIf, FrIf, EthIf).
- Bus interfaces communicate with MCAL drivers: CanDriver, LinDriver, FlexRayDriver, EthDriver.
- Dcm (Diagnostic Communication Manager) interfaces with PduR for diagnostic requests.

Overview of CP Software Clusters: Software Cluster Connection (SwCluC)

The module Software Cluster Connection (SwCluC) consists of:

1. Cross Software Cluster Communication (SwCluC_Xcc): Connects software clusters based on Binary Manifest (BManif).
2. Proxy Modules: High Proxies substitute non-local BSW in Application Software Clusters; Low Proxies connect to regular BSW in the Host Software Cluster (OS, NvM, Dem, Dcm proxies).
3. Binary Manifest (BManif): Provides binary meta information for interfaces connecting clusters.

Integration and Runtime Aspects: Mapping of Runnables

- Runnables are the active executing parts of Software Components.
- Runnables are mapped to OS Tasks for concurrent execution.
- Entity hierarchy: Runnable -> Task -> OS-Application -> Partition -> uC-Core.
- BSW resources (e.g. NV-blocks) are accessed across partitions via RTE and BSW Scheduler.

Integration and Runtime Aspects: Secure Onboard Communication (SecOC)

SecOC provides message authentication and freshness verification:

- SecOC verifies and generates Message Authentication Codes (MAC) and Freshness Values (FV).
- SecOC BSW module is integrated between PduR and lower-layer communication interfaces (CanIf, CanTp, FrIf, FrTp).
- Interacts with Crypto Service Manager (Csm) and Key & Counter Management SW-Cs.

Integration and Runtime Aspects: Global Time Synchronization

StbM (Synchronized Time-base Manager) provides:

- Synchronized time base over multiple in-vehicle networks.
- Protocol providers: CanTSyn (CAN), FrTSyn (FlexRay), EthTSyn (Ethernet with rate-correction and latency calculation).
- Primary use cases: Sensor data fusion and Cross-ECU logging.